



МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»



**НГТУ
НЭТИ** | **Факультет прикладной
математики и информатики**

Кафедра теоретической и прикладной информатики
Лабораторная работа № 3
по дисциплине «Методы оптимизации»

МЕТОД ШТРАФНЫХ ФУНКЦИЙ

Бригада 9	ГРУШЕВ АНДРЕЙ
Группа ПМ-05	ХАБАРОВА АНАСТАСИЯ
Вариант 9	

Преподаватели	ЛЕМЕШКО БОРИС ЮРЬЕВИЧ
	ФИЛИППОВА ЕЛЕНА ВЛАДИМИРОВНА

Новосибирск, 2023

Цель работы

Ознакомиться с методами штрафных функций при решении задач нелинейного программирования. Изучить типы штрафных и барьерных функций, их особенности, способы и области применения, влияние штрафных функций на сходимость алгоритмов, зависимость точности решения задачи нелинейного программирования от величины коэффициента штрафа.

Задание

- Применяя методы поиска минимума 0-го порядка, реализовать программу для решения задачи нелинейного программирования с использованием метода штрафных функций.
- Исследовать сходимость метода штрафных функций в зависимости от
 - выбора штрафных функций,
 - начальной величины коэффициента штрафа,
 - стратегии изменения коэффициента штрафа,
 - начальной точки,
 - задаваемой точности.
- Сформулировать выводы.

Первая задача (a)	Вторая задача (b)
$f(x, y) = 4(x + y)^2 + x^2 \rightarrow \min$ $y - x \geq 5$	$f(x, y) = 4(x + y)^2 + x^2 \rightarrow \min$ $x = 2 - y$
Решение: $f(-2.35294117, 2.64705882) = 5.88235294$	Решение: $f(0, 2) = 16$

Результаты исследований

Задача а:

$$Q_{0_1} = G(g(\bar{x})) = 0.5(g(\bar{x}) + |g(\bar{x})|)$$

$$Q_{0_2} = G(g(\bar{x})) = (0.5(g(\bar{x}) + |g(\bar{x})|))^2$$

$$Q_{0_3} = G(g(\bar{x})) = (0.5(g(\bar{x}) + |g(\bar{x})|))^4$$

$$r_i = r_{i-1} * 2$$

	r_0	x_0	y_0	ε	iters	f_calc	x^*	y^*	$f(x^*, y^*)$	df
Q_0_1	1	0	0	1.E-03	2	1077	-2.361168	2.638851	5.883546	1.193345E-03
Q_0_2					11	2632	-2.353123	2.646304	5.881681	-6.722736E-04
Q_0_3					30	6676	-2.352483	2.646689	5.880910	-1.443350E-03
Q_0_1				1.E-04	2	1429	-2.362005	2.637989	5.883760	1.406741E-03
Q_0_2					14	4253	-2.352912	2.647016	5.882268	-8.447577E-05
Q_0_3					40	11232	-2.352791	2.647127	5.882210	-1.430252E-04
Q_0_1				1.E-05	2	1771	-2.362030	2.637971	5.883759	1.405900E-03
Q_0_2					17	6249	-2.352938	2.647053	5.882342	-1.055821E-05
Q_0_3					50	16891	-2.352932	2.647060	5.882339	-1.421248E-05
Q_0_1				1.E-06	2	2132	-2.362032	2.637968	5.883758	1.405000E-03
Q_0_2					21	9064	-2.352940	2.647060	5.882352	-6.595342E-07
Q_0_3					60	24011	-2.352942	2.647057	5.882352	-1.410060E-06
Q_0_1				1.E-07	2	2501	-2.362032	2.637968	5.883758	1.404977E-03
Q_0_2					24	12092	-2.352941	2.647059	5.882353	-8.245812E-08
Q_0_3					69	32079	-2.352941	2.647059	5.882353	-1.762512E-07

$$r_i = r_{i-1} * 5$$

	r_0	x_0	y_0	ε	$iters$	f_calc	x^*	y^*	$f(x^*, y^*)$	df
Q_0_1	1	0	0	1.E-03	1	679	-2.37507	2.624934	5.890696	8.342665E-03
Q_0_2					5	1405	-2.35225	2.647366	5.881915	-4.381785E-04
Q_0_3					13	3012	-2.3518	2.647382	5.880983	-1.369740E-03
Q_0_1				1.E-04	1	911	-2.37518	2.624825	5.890768	8.415311E-03
Q_0_2					6	2140	-2.35286	2.647067	5.882264	-8.853254E-05
Q_0_3					17	4935	-2.35288	2.647026	5.882191	-1.617326E-04
Q_0_1				1.E-05	1	1247	-2.37405	2.625946	5.889932	7.579228E-03
Q_0_2					8	3277	-2.35294	2.647056	5.882349	-3.533758E-06
Q_0_3					22	7631	-2.35292	2.647069	5.882342	-1.104786E-05
Q_0_1				1.E-06	1	1378	-2.37517	2.624833	5.890751	8.397734E-03
Q_0_2					9	4411	-2.35294	2.647059	5.882352	-7.084396E-07
Q_0_3					26	10614	-2.35294	2.647057	5.882352	-1.294441E-06
Q_0_1				1.E-07	1	1609	-2.37517	2.624833	5.890751	8.397675E-03
Q_0_2					11	6057	-2.35294	2.647059	5.882353	-2.833009E-08
Q_0_3					30	14195	-2.35294	2.647059	5.882353	-1.514101E-07

$$r_i = r_{i-1} * 100$$

	r_0	x_0	y_0	ε	$iters$	f_calc	x^*	y^*	$f(x^*, y^*)$	df
Q_0_1	1	0	0	1.E-03	1	544	-2.35435	2.645659	5.882398	4.500912E-05
Q_0_2					2	758	-2.35305	2.646847	5.882218	-1.348135E-04
Q_0_3					5	1289	-2.35389	2.645751	5.881697	-6.561027E-04
Q_0_1				1.E-04	1	730	-2.35314	2.64686	5.882357	4.268579E-06
Q_0_2					3	1270	-2.3531	2.646898	5.882355	1.926886E-06
Q_0_3					6	1938	-2.35282	2.647094	5.882205	-1.476231E-04
Q_0_1				1.E-05	1	895	-2.35297	2.647032	5.882353	4.549136E-07
Q_0_2					3	1569	-2.35294	2.647063	5.882352	-1.357170E-06
Q_0_3					8	3009	-2.35295	2.647049	5.882346	-6.859932E-06
Q_0_1				1.E-06	1	1066	-2.35294	2.647057	5.882353	5.103811E-08
Q_0_2					4	2254	-2.35294	2.647057	5.882353	3.332801E-08
Q_0_3					9	3968	-2.35294	2.647056	5.882351	-1.478492E-06
Q_0_1				1.E-07	1	1244	-2.35294	2.647059	5.882353	2.079975E-09
Q_0_2					4	2700	-2.35294	2.647059	5.882353	-1.349663E-08
Q_0_3					11	5545	-2.35294	2.647059	5.882353	-6.843329E-08

$$r_i = r_{i-1} * 200$$

	r_0	x_0	y_0	ε	$iters$	f_calc	x^*	y^*	$f(x^*, y^*)$	df
Q_0_1	1	0	0	1.E-03	1	544	-2.35485	2.64516	5.88243	7.698031E-05
Q_0_2					2	758	-2.35276	2.647192	5.882327	-2.612997E-05
Q_0_3					4	1068	-2.35427	2.645053	5.881141	-1.212374E-03
Q_0_1				1.E-04	1	730	-2.35314	2.64686	5.882357	4.266603E-06
Q_0_2					2	1004	-2.35279	2.647184	5.882319	-3.426398E-05
Q_0_3					6	1934	-2.35319	2.646788	5.882317	-3.561482E-05
Q_0_1				1.E-05	1	895	-2.35297	2.647035	5.88236	7.112452E-06
Q_0_2					3	1568	-2.35297	2.647028	5.882353	-5.334341E-08

Q_0_3				1.E-06	7	2661	-2.35295	2.647049	5.882347	-6.303460E-06
Q_0_1					1	1066	-2.35294	2.647057	5.882353	4.680505E-08
Q_0_2					3	1874	-2.35294	2.647059	5.882353	-1.719593E-07
Q_0_3					8	3574	-2.35294	2.647057	5.882352	-1.080390E-06
Q_0_1				1.E-07	1	1248	-2.35294	2.647058	5.882353	7.055619E-09
Q_0_2					4	2679	-2.35294	2.647059	5.882353	-6.165370E-10
Q_0_3					10	5053	-2.35294	2.647059	5.882353	-3.160492E-08

При различных стратегиях изменения параметра r лучше всего себя показала функция Q_0_2 в среднем имея лучшую точность и немного уступая функции Q_0_1 по количеству вычислений целевой функции.

Лучшей стратегией изменения параметра r оказалась $r_i = r_{i-1} * 200$, так как при её применении удалось добиться лучшей точности при наименьшем количестве вычислений целевой функции.

Проведём дополнительные исследования относительно r_0 при $r_i = r_{i-1} * 200$.

	r_0	x_0	y_0	ε	$iters$	f_calc	x^*	y^*	$f(x^*, y^*)$	df
Q_0_1	1	0	0	1.00E-07	1	1248	-2.35294	2.647058	5.882353	7.055619E-09
Q_0_2					4	2679	-2.35294	2.647059	5.882353	-6.165370E-10
Q_0_3					10	5053	-2.35294	2.647059	5.882353	-3.160492E-08
Q_0_1	2				1	1290	-2.35294	2.647058	5.882353	5.584583E-09
Q_0_2					3	2105	-2.35294	2.647059	5.882353	-8.649464E-08
Q_0_3					9	4585	-2.35294	2.647059	5.882353	-1.466220E-07
Q_0_1	5				0	762	-2.37517	2.624833	5.890751	8.397675E-03
Q_0_2					3	1959	-2.35294	2.647059	5.882353	-3.454515E-08
Q_0_3					9	4527	-2.35294	2.647059	5.882353	-1.080785E-07
Q_0_1	10				0	1038	-2.35294	2.647059	5.882353	5.845824E-08
Q_0_2					3	1887	-2.35294	2.647059	5.882353	-1.703438E-08
Q_0_3					9	4512	-2.35294	2.647059	5.882353	-8.578031E-08
Q_0_1	100	0	397	-2.35294	2.647059	5.882353	2.079975E-09			
Q_0_2		3	1766	-2.35294	2.647059	5.882353	-1.635368E-09			
Q_0_3		9	4464	-2.35294	2.647059	5.882353	-3.979939E-08			
Q_0_1	200	0	401	-2.35294	2.647058	5.882353	7.055619E-09			
Q_0_2		3	1751	-2.35294	2.647059	5.882353	-6.165370E-10			
Q_0_3		9	4447	-2.35294	2.647059	5.882353	-3.160492E-08			
Q_0_1	1000	0	401	-2.35294	2.647058	5.882353	5.554566E-09			
Q_0_2		2	1291	-2.35294	2.647059	5.882353	-3.454515E-08			
Q_0_3		8	4004	-2.35294	2.647059	5.882353	-1.080785E-07			

При $r_0 \geq 5$ функция Q_0_1 не проводит ни одной итерации, а при $r_0 = 5$ теряется большая часть точности. В остальном выбор r_0 на работу функций почти не влияет.

Проведём дополнительные исследования относительно начальной точки.

	r_0	x_0	y_0	ε	$iters$	f_calc	x^*	y^*	$f(x^*, y^*)$	df
Q_0_1	1	0	0	1.00E-07	1	1248	-2.35294	2.647058	5.882353	7.055619E-09
Q_0_2		0	0		4	2679	-2.35294	2.647059	5.882353	-6.165370E-10
Q_0_3		0	0		10	5053	-2.35294	2.647059	5.882353	-3.160492E-08
Q_0_1		10	10		1	1597	-2.35294	2.647055	5.882353	7.200125E-08
Q_0_2		10	10		4	4581	-2.35296	2.647045	5.882353	6.425563E-08
Q_0_3		10	10		10	10026	-2.35294	2.647062	5.882353	1.001631E-08
Q_0_1		-10	-10		1	2940	-2.54398	2.456023	6.502764	6.204110E-01
Q_0_2		-10	-10		4	7620	-2.35228	2.64772	5.88236	7.439235E-06
Q_0_3		-10	-10		10	18159	-2.35228	2.64772	5.88236	7.400085E-06
Q_0_1		100	100		1	1843	-2.36801	2.631988	5.886214	3.861513E-03
Q_0_2		100	100		4	5094	-2.35294	2.647059	5.882353	2.648406E-08
Q_0_3		100	100		10	11116	-2.35293	2.647073	5.882353	2.533733E-08
Q_0_1		-100	-100		1	2399	-2.33879	2.661209	5.885757	3.403731E-03
Q_0_2		-100	-100		3	31240	-2.32595	2.674052	5.894739	1.238654E-02
Q_0_3		-100	-100		10	78080	-2.32593	2.674068	5.894754	1.240132E-02

При удалении начальной точки от решения наблюдается потеря точности для всех функций.

Задача б:

$$Q_{1_0} = H(h(\bar{x})) = |h(\bar{x})|$$

$$Q_{2_0} = H(h(\bar{x})) = h(\bar{x})^2$$

$$Q_{3_0} = H(h(\bar{x})) = h(\bar{x})^4$$

$$r_i = r_{i-1} * 2$$

	r_0	x_0	y_0	ε	iters	f_calc	x^*	y^*	$f(x^*, y^*)$	df
Q_1_0	1	0	0	1.E-03	4	1316	-2.47E-05	1.999548	16	9.085945E-07
Q_2_0					13	3014	1.998935	0.000146	19.98796	3.987960E+00
Q_3_0					32	5690	1.998935	0.000146	19.98411	3.984105E+00
Q_1_0				1.E-04	4	1890	7.12E-05	1.999884	16	1.296583E-08
Q_2_0					17	4912	1.999922	1.46E-05	19.9992	3.999199E+00
Q_3_0					42	9845	1.999898	1.46E-05	19.99845	3.998449E+00
Q_1_0				1.E-05	4	2427	-3.58E-07	1.999995	16	9.553602E-11
Q_2_0					20	7191	1.99999	1.46E-06	19.9999	3.999901E+00
Q_3_0					52	15209	1.99999	1.46E-06	19.99985	3.999850E+00
Q_1_0				1.E-06	4	3010	5.66E-08	2	16	8.526513E-14
Q_2_0					24	10231	1.999999	1.46E-07	19.99999	3.999994E+00
Q_3_0					63	22079	1.999999	1.46E-07	19.99999	3.999987E+00
Q_1_0				1.E-07	4	3559	2.56E-09	2	16	1.065814E-14
Q_2_0					27	13019	2	1.46E-08	20	3.999999E+00
Q_3_0					73	29401	2	1.46E-08	20	3.999999E+00

$$r_i = r_{i-1} * 5$$

	r_0	x_0	y_0	ε	iters	f_calc	x^*	y^*	$f(x^*, y^*)$	df
Q_1_0	1	0	0	1.E-03	2	568	1.999796	0.000146	19.99971	3.999709E+00
Q_2_0					6	1429	1.999264	0.000146	19.99306	3.993057E+00
Q_3_0					14	2465	1.998935	0.000146	19.98539	3.985395E+00
Q_1_0				1.E-04	2	833	1.999998	-1.46E-05	20.00014	4.000140E+00
Q_2_0					8	2336	1.99996	1.46E-05	19.99969	3.999686E+00
Q_3_0					18	4451	1.999898	1.46E-05	19.99841	3.998415E+00
Q_1_0				1.E-05	2	1083	1.999998	1.46E-06	20	3.999999E+00
Q_2_0					9	3368	1.999996	1.46E-06	19.99995	3.999955E+00
Q_3_0					23	6727	1.99999	1.46E-06	19.99988	3.999884E+00
Q_1_0				1.E-06	2	1362	2	-1.46E-07	20	4.000002E+00
Q_2_0					10	4333	1.999999	1.46E-07	19.99999	3.999990E+00
Q_3_0					27	9551	1.999999	1.46E-07	19.99999	3.999987E+00
Q_1_0				1.E-07	2	1634	2	1.46E-08	20	4.000000E+00
Q_2_0					12	6043	2	-1.46E-08	20	4.000000E+00
Q_3_0					32	13112	2	1.46E-08	20	3.999999E+00

$$r_i = r_{i-1} * 100$$

	r_0	x_0	y_0	ε	iters	f_calc	x^*	y^*	$f(x^*, y^*)$	df
Q_1_0	1	0	0	1.E-03	1	267	1.999796	0.000146	20.0041	4.004105E+00
Q_2_0					2	557	1.998935	0.000146	19.98949	3.989487E+00
Q_3_0					5	1031	1.999264	0.000146	19.98883	3.988827E+00
Q_1_0				1.E-04	1	402	1.999998	-1.46E-05	20.00136	4.001365E+00
Q_2_0					3	1045	1.999998	-1.46E-05	20	3.999998E+00
Q_3_0					7	1676	1.99996	1.46E-05	19.99947	3.999474E+00
Q_1_0				1.E-05	1	526	2	-1.46E-06	20.00009	4.000090E+00
Q_2_0					3	1306	1.99999	1.46E-06	19.9999	3.999897E+00
Q_3_0					8	2576	1.99999	1.46E-06	19.99988	3.999875E+00
Q_1_0				1.E-06	1	688	2	1.46E-07	20	4.000004E+00
Q_2_0					4	1882	2	1.46E-07	20	4.000000E+00
Q_3_0					10	3584	1.999999	1.46E-07	19.99999	3.999994E+00
Q_1_0				1.E-07	1	818	2	-1.46E-08	20	4.000001E+00
Q_2_0					4	2288	2	6.18E-08	20	3.999999E+00
Q_3_0					11	4783	2	1.46E-08	20	3.999999E+00

$$r_i = r_{i-1} * 200$$

	r_0	x_0	y_0	ε	iters	f_calc	x^*	y^*	$f(x^*, y^*)$	df
Q_1_0	1	0	0	1.E-03	1	267	1.999796	0.000146	20.00996	4.009965E+00
Q_2_0					2	573	1.999796	0.000146	19.99838	3.998382E+00
Q_3_0					5	915	1.999796	0.000146	19.99825	3.998248E+00
Q_1_0				1.E-04	1	402	1.999998	-1.46E-05	20.003	4.002998E+00
Q_2_0					3	832	1.999998	1.46E-05	20.00152	4.001519E+00
Q_3_0					6	1549	1.99996	1.46E-05	19.99946	3.999459E+00
Q_1_0				1.E-05	1	526	2	-1.46E-06	20.00019	4.000195E+00
Q_2_0					3	1362	2.000001	-1.46E-06	20	4.000005E+00

Q_3_0					7	2241	1.99999	1.46E-06	19.99989	3.999888E+00
Q_1_0				1.E-06	1	688	2	1.46E-07	20.00001	4.000007E+00
Q_2_0					4	1644	2	1.46E-07	20	4.000002E+00
Q_3_0					9	3152	2	-1.46E-07	20	3.999998E+00
Q_1_0				1.E-07	1	818	2	-1.46E-08	20	4.000002E+00
Q_2_0					4	2154	2	-1.46E-08	20	4.000000E+00
Q_3_0					10	4153	2	-1.46E-08	20	4.000000E+00

При различных стратегиях изменения параметра r лучше всего себя показала функция Q_1_0 имея лучшую точность и наименьшее количество вычислений целевой функции. Лучшей стратегией изменения параметра r оказалась $r_i = r_{i-1} * 2$, так как при её применении удалось добиться лучшей точности.

Проведём дополнительные исследования относительно r_0 при $r_i = r_{i-1} * 2$.

	r_0	x_0	y_0	ε	$iters$	f_calc	x^*	y^*	$f(x^*, y^*)$	df
Q_1_0	1	0	0	1.00E-07	4	3559	2.56E-09	2	16	1.065814E-14
Q_2_0					27	13019	2	1.46E-08	20	3.999999E+00
Q_3_0					73	29401	2	1.46E-08	20	3.999999E+00
Q_1_0	2				3	2843	2.56E-09	2	16	1.065814E-14
Q_2_0					26	12256	2	1.46E-08	20	3.999999E+00
Q_3_0					72	28678	2	1.46E-08	20	3.999999E+00
Q_1_0	5				2	1775	2	1.46E-08	20	4.000000E+00
Q_2_0					25	11115	2	1.46E-08	20	3.999999E+00
Q_3_0					70	27588	2	-1.46E-08	20	3.999999E+00
Q_1_0	10				1	959	2	1.46E-08	20	4.000000E+00
Q_2_0					24	10355	2	1.46E-08	20	3.999999E+00
Q_3_0					69	26898	2	-1.46E-08	20	3.999999E+00
Q_1_0	100	0	102	2	-1.46E-08	20	4.000001E+00			
Q_2_0		20	7996	2	-1.46E-08	20	3.999999E+00			
Q_3_0		66	24620	2	-1.46E-08	20	3.999999E+00			
Q_1_0	200	0	102	2	-1.46E-08	20	4.000002E+00			
Q_2_0		19	7377	2	-1.46E-08	20	3.999999E+00			
Q_3_0		65	23989	2	-1.46E-08	20	3.999999E+00			
Q_1_0	1000	0	102	2	-1.46E-08	20.00001	4.000013E+00			
Q_2_0		17	6348	2	1.46E-08	20	3.999999E+00			
Q_3_0		63	23080	2	1.46E-08	20	3.999999E+00			

Функция Q_1_0 даёт точное решение, а при $r_0 \leq 2$.

Проведём дополнительные исследования относительно начальной точки.

	r_0	x_0	y_0	ε	iters	f_calc	x^*	y^*	$f(x^*, y^*)$	df
Q_1_0	1	0	0	1.00E-07	4	3559	2.56E-09	2	16	1.065814E-14
Q_2_0		0	0		27	13019	2	1.46E-08	20	3.999999E+00
Q_3_0		0	0		73	29401	2	1.46E-08	20	3.999999E+00
Q_1_0		10	10		4	3458	-8	10	80	6.400000E+01
Q_2_0		10	10		27	14334	-8	10	80	6.400000E+01
Q_3_0		10	10		72	36542	-8	10	80	6.400000E+01
Q_1_0		-10	-10		4	3771	-1.30E-08	2	16	3.552714E-15
Q_2_0		-10	-10		27	14473	12	-10	160	1.440000E+02
Q_3_0		-10	-10		73	33792	12	-10	160	1.440000E+02
Q_1_0		100	100		4	4518	9.13E-09	2	16	7.105427E-15
Q_2_0		100	100		27	16200	-3.14E-08	2	16	-4.289576E-07
Q_3_0		100	100		71	42149	5.63E-08	2	16	-1.350138E-06
Q_1_0		-100	-100		4	4415	8.18E-08	2	16	7.105427E-15
Q_2_0		-100	-100		27	16564	-1.89E-08	2	16	-4.278774E-07
Q_3_0		-100	-100		71	39862	-4.11E-08	2	16	-1.354147E-06

При удалении начальной точки от решения наблюдается увеличение точности для всех функций.

Выводы

При использовании метода штрафных функций точность в наибольшей степени зависит от выбора штрафной функции, стратегии изменения параметра r и начальной точки.

Для данной задачи было установлено, что лучшими функциями являются $H(h(\bar{x})) = |h(\bar{x})|$ и $G(g(\bar{x})) = (0.5(g(\bar{x}) + |g(\bar{x})|))^2$, стратегии $r_i = r_{i-1} * 2$ и $r_i = r_{i-1} * 200$ соответственно.

Код программы

main.py

```
import numpy as np
from math import log
import csv

alpha: int = 4
r: float = 1
func_count: int = 0

def func_f(x: np.array) -> float:
    global func_count
    func_count += 1
    return 4*(x[0, 0] + x[0, 1])**2 + x[0, 0]**2

def func_g(x: np.array) -> float:
    return x[0, 0] - x[0, 1] + 5

def func_h(x: np.array) -> float:
    return x[0, 0] - 2 + x[0, 1]

def H1(x: np.array) -> float:
    return abs(func_h(x))

def H2(x: np.array) -> float:
    return abs(func_h(x)) ** 2

def H3(x: np.array) -> float:
    return abs(func_h(x)) ** alpha

def G1(x: np.array) -> float:
    value = func_g(x)
    return (value + abs(value)) / 2

def G2(x: np.array) -> float:
    value = func_g(x)
    return ((value + abs(value)) / 2) ** 2

def G3(x: np.array) -> float:
    value = func_g(x)
    return ((value + abs(value)) / 2) ** alpha

def G4(x: np.array) -> float:
    return -(1 / func_g(x))

def G5(x: np.array) -> float:
    return -log(-func_g(x))

#Функции для метода штрафной функций
```

```

def Q_0_1(x: np.array) -> float:
    return(func_f(x) + r * (G1(x)))

def Q_0_2(x: np.array) -> float:
    return(func_f(x) + r * (G2(x)))

def Q_0_3(x: np.array) -> float:
    return(func_f(x) + r * (G3(x)))

def Q_1_0(x: np.array) -> float:
    return(func_f(x) + r * (H1(x)))

def Q_2_0(x: np.array) -> float:
    return(func_f(x) + r * (H2(x)))

def Q_3_0(x: np.array) -> float:
    return(func_f(x) + r * (H3(x)))

#####

#Функции для метода барьерных функций

def Q_0_4(x: np.array) -> float:
    return(func_f(x) + r * (G4(x)))

def Q_0_5(x: np.array) -> float:
    return(func_f(x) + r * (G5(x)))

#####

def min_interval(begin: float, func: callable, delta: float, x: np.array, s: np.ar-
ray) -> tuple[float, float]:
    """
    Finds an interval for search the minimum of a function

    :param begin: start point for searching
    :param func: callable function
    :param delta: precision for first iteration

    :return: void
    """

    iters = h = 0
    prev = curr = next = begin

    if (func(x + begin * s) > func(x + (begin + delta) * s)):
        next = begin + delta
        h = delta
    if (func(x + begin * s) > func(x + (begin - delta) * s)):
        next = begin - delta

```

```

        h = -delta

    if (h != 0):
        while (func(x + curr * s) > func(x + next * s)):
            iters += 1
            prev = curr
            curr = next

            h *= 2
            next = curr + h

        return prev, next
    else:
        return begin - delta, begin + delta

def golden_section(begin: int, end: int, func: callable, eps: float, x: np.array,
s: np.array) -> float:
    """
    Finds an interval that contains the minimum of a function
    Using a Golden Section Algorithm

    :param begin: begin of initial interval
    :param end: end of initial interval
    :param func: callable function (x, y)
    :param eps: precision

    :return: void
    """

    golden_number = 0.381966011

    iters = 0

    left = begin + golden_number * (end - begin)
    right = end - golden_number * (end - begin)

    value_left = func(x + left * s)
    value_right = func(x + right * s)

    while (abs(end - begin) > eps):
        iters += 1

        if (value_left < value_right):
            end = right
            right = left
            value_right = value_left
            left = begin + golden_number*(end - begin)
            value_left = func(x + left * s)
        else:
            begin = left

```

```

        left = right
        value_left = value_right
        right = end - golden_number*(end - begin)
        value_right = func(x + right * s)

    return (begin + end) / 2

def rosenbrock(x: np.array, eps: float, target: callable):

    s = np.matrix([[1, 0], [0, 1]], dtype = float)
    a = np.matrix([[0, 0], [0, 0]], dtype = float)

    deltaF: float = eps
    deltaX: bool = True

    while (deltaF >= eps and deltaX == True):

        x_prev = x

        begin, end = min_interval(0, target, eps, x, s[0])
        lambda1: float = golden_section(begin, end, target, eps, x, s[0])

        x = x + s[0] * lambda1

        begin, end = min_interval(0, target, eps, x, s[1])
        lambda2: float = golden_section(begin, end, target, eps, x, s[1])

        x = x + s[1] * lambda2

        a[0] = lambda1 * s[0] + lambda2 * s[1]

        if (abs(lambda1) < abs(lambda2)):
            a[1] = lambda1 * s[1]
        else:
            a[1] = lambda2 * s[1]

        s[0] = a[0] / np.linalg.norm(a[0])

        b = a[1] - (a[1] * s[0].transpose()) * s[0]

        s[1] = b / np.linalg.norm(b)

        for i in range(2):
            if (abs(np.asmatrix(x)[0, i] - np.asmatrix(x_prev)[0, i]) < eps):
                deltaX = False
                break

        deltaF = abs(target(np.asmatrix(x)) - target(np.asmatrix(x_prev)))

    return x

```

```
#####

def main():
    # 1 r *= 2
    # 2 r *= 5
    # 3 r *= 100
    # 4 r *= 200

    A_solve = 5.8823529411764705882352941176471
    B_solve = 16.0
    global r
    global func_count

    # targets = [Q_0_1, Q_0_2, Q_0_3]
    # eps_range = [1e-3, 1e-4, 1e-5, 1e-6, 1e-7]

    #target: callable = Q_0_1
    #x = np.array([0, 0], dtype = float)
    # eps: float = 1e-7
    #r0_range = [1, 2, 5, 10, 100, 200, 1000]
    # r0 = 1

    # x_range = [
    #     np.array([0, 0], dtype = float),
    #     np.array([10, 10], dtype = float),
    #     np.array([-10, -10], dtype = float),
    #     np.array([100, 100], dtype = float),
    #     np.array([-100, -100], dtype = float),
    #     ]

    # with open(f'x0.csv', 'w', newline='') as csvfile:
    #     spamwriter = csv.writer(csvfile, delimiter=';', quotechar='|', quoting=csv.QUOTE_MINIMAL)
    #     for x in x_range:
    #         for target in targets:
    #             r = r0
    #             iters = 0
    #             func_count = 0

    #             solve = rosenbrock(x, eps, target)

    #             while (func_g(solve) > eps):
    #                 #r *= 2
    #                 #r *= 5
    #                 #r *= 100
    #                 r *= 200
    #                 iters += 1
    #                 solve = rosenbrock(x, eps, target)
```

```

#             spamwriter.writerow([target.__name__, r0, x[0], x[1], eps, iters,
func_count, solve[0, 0], solve[0, 1], target(solve), target(solve) - A_solve])

# 1 r *= 2
# 2 r *= 5
# 3 r += 10
# 4 r += 100

targets = [Q_1_0, Q_2_0, Q_3_0]
eps_range = [1e-3, 1e-4, 1e-5, 1e-6, 1e-7]

x = np.array([0, 0], dtype = float)
eps: float = 1e-7
r0_range = [1, 2, 5, 10, 100, 200, 1000]
r0 = 1

x_range = [
    np.array([0, 0], dtype = float),
    np.array([10, 10], dtype = float),
    np.array([-10, -10], dtype = float),
    np.array([100, 100], dtype = float),
    np.array([-100, -100], dtype = float),
]

with open(f'x0.csv', 'w', newline='') as csvfile:
    spamwriter = csv.writer(csvfile, delimiter=';', quotechar='|', quoting=csv.QUOTE_MINIMAL)
    for x in x_range:
        for target in targets:
            r = r0
            iters = 0
            func_count = 0

            solve = rosenbrock(x, eps, target)

            while (abs(func_h(solve)) > eps):
                r *= 2
                #r *= 5
                #r *= 100
                #r *= 200
                iters += 1
                solve = rosenbrock(x, eps, target)

            spamwriter.writerow([target.__name__, r0, x[0], x[1], eps, iters,
func_count, solve[0, 0], solve[0, 1], target(solve), target(solve) - B_solve])

if __name__ == "__main__":
    main()

```